

EPIC playground

ANÁLISIS Y DISEÑO DE ALGORITMOS

ALDAMA NAVA NADIR IBRAIN
PERALTA LLERA ELIZABETH
TORIBIO SEGURA ALMA JESSICA
VARILLAS FIGUEROA ENRIQUE URIEL

CONTENIDO

1. HERRAMIENTAS UTILIZADAS
2. ESTRUCTURA DEL PROYECTO
3. DOMINIO
4. EDITOR
5. MOTOR
6. SIMULADOR
7. VISUALIZADOR

HERRAMIENTAS UTILIZADAS



Claude



python

ESTRUCTURA DEL PROYECTO

DOMINIO

Repositorio estructural que garantiza la coherencia entre entidades, relaciones y atributos visuales del sistema.

EDITOR

Responsable de la captura interactiva y la traducción de la entrada del usuario a estructuras de datos validadas.

MOTOR

Ejecuta el cálculo matricial de propagación para alcanzar los estados de estabilización lógica

SIMULADOR

Analiza el estado estabilizado y orquesta la secuencia de eventos de ejecución definidos.

VISUALIZADOR

Mantiene la sincronización entre el estado lógico del sistema y su manifestación visual, garantiz

SUBEQUIPO 6

SIMULADOR

DEFINICIÓN: Su función es tomar los resultados matemáticos del motor y convertirlos en un conjunto de pasos o acciones ordenadas que el sistema debe seguir.

RESPONSABILIDADES:

- Gestión de tiempos: Controlar en que momento debe ocurrir cada evento dentro de la simulación.
- Interpretación. Traducir los datos técnicos (matrices/estados) en instrucciones de acción claras para el sistema.
- Control de flujo: Asegurar que la simulación ocurra siguiendo la lógica definida sin saltos ni errores de orden.

SUBEQUIPO 6

VISUALIZADOR

DEFINICIÓN: Se encarga de traducir los datos y eventos técnicos en elementos gráficos que el usuario puede observar y entender sin esfuerzo.

RESPONSABILIDADES:

- Renderizado: Dibujar en pantalla los nodos, conexiones y estados definidos en el Dominio Compartido.
- Sincronización: Mantener la pantalla actualizada en tiempo real con los cambios que ocurren en el backend.
- Interactividad visual: Aplicar efectos visuales (colores, animaciones) cuando el sistema cambia de estado o ejecuta un evento.

SUBEQUIPO 6

METODOLOGÍA DE TRABAJO

Para asegurar la mejor calidad, el subequipo 6 utilizó la siguiente metodología:

- DESARROLLO PARALELO: Cada integrante generó prompts distintos para los mismos módulos.
- SELECCIÓN CUALITATIVA: Se evaluaron resultados y se eligieron los que cumplieran mejor con el objetivo.
- VALIDACIÓN TÉCNICA: Se realizaron pruebas de integración para asegurar que los datos del editor fueran legibles por el motor y procesables por el simulador.

6. EL SIMULADOR: LA MEMORIA DEL SISTEMA

El simulador administra la ejecución.

recuerda los resultados y te deja navegar entre ellos.

Guarda:

- estados
- historial
- iteraciones
- resultados

ALMACEN DE MEMORIA

```
class GestorEstadoSimulacion:
    def __init__(self) -> None:
        self._estados: dict[str, EstadoSimulacion] = {}
        #
        # ID único estado guardado

    def crear(self) -> tuple[str, EstadoSimulacion]:
        sim_id = str(uuid.uuid4()) # ID único generado automáticamente
        estado = EstadoSimulacion(fase='inactivo')
        self._estados[sim_id] = estado
        return sim_id, estado

    def avanzar_paso(self, sim_id: str) -> bool:
        estado = self.obtener(sim_id)
        if estado.paso_actual < len(estado.historial) - 1:
            estado.paso_actual += 1
            return True # sí avanzó
        return False # ya estaba en el último paso
```

¿Que guarda?

- Estado actual
- Historial
- Fase

Limitación: todo está en memoria. Si reinicias el servidor, se pierde todo.

SERVICIO.PY — EL DIRECTOR DE ORQUESTA

Cuando el usuario solicita una simulación, el servicio realiza las siguientes actividades:

1. Valida la red lógica.
2. Crea una nueva simulación.
3. Cambia el estado a "Corriendo".
4. Solicita al Motor que realice los cálculos.
5. Guarda el historial generado.
6. Actualiza el estado a "Completado".
7. Devuelve el identificador de la simulación.

```
class SimulacionServicio(ISimulacion):  
  
    def __init__(self, fabrica_motor: FabricaMotor,  
                  gestor: GestorEstadoSimulacion) -> None:  
        self._fabrica_motor = fabrica_motor # ← NO es el motor en sí,  
        self._gestor = gestor               # es una función que lo crea
```

Todos los pasos quedan calculados inmediatamente.

Gracias a esto, cuando el usuario presiona los botones de avanzar o retroceder, no es necesario volver a ejecutar el algoritmo, lo que mejora considerablemente el rendimiento del sistema.

```
def iniciar(self, red: Red) -> str:
    errores = validar_red(red)
    if errores: raise RedInvalidaError(...)

    sim_id, _ = self._gestor.crear()
    self._gestor.actualizar_fase(sim_id, FaseSimulacion.CORRIENDO)

    motor = self._fabrica_motor()          # crear el motor
    pasos = motor.calcular_pasos(red)      # calcular TODO el historial

    self._gestor.establecer_historial(sim_id, pasos)

    ultimo = pasos[-1]
    fase_final = ERROR if ultimo.error else COMPLETADO
    self._gestor.actualizar_fase(sim_id, fase_final)

    return sim_id
```

API.PY DEL SIMULADOR — COMPOSITION ROOT

```
from motor.algoritmos import obtener_motor

_servicio = SimulacionServicio(
    fabrica_motor=lambda: obtener_motor(),    # ← aquí se conectan
    gestor=_gestor                           # Motor y Simulador
)
```

Funciona como la interfaz de comunicación entre el Frontend y el Simulador.

Funciones principales

- Recibir datos en formato JSON.
- Validar el formato de entrada.
- Invocar al Simulador.
- Devolver respuestas en formato JSON.

7. PRINCIPAL.PY: PUNTO DE ARRANQUE DEL SISTEMA

```
app = FastAPI(...)
app.add_middleware(CORSMiddleware, allow_origins=['*'], ...)
app.include_router(router_motor)
app.include_router(router_simulador)
```

El archivo principal.py es el punto de entrada del Backend. Cuando el servidor inicia, este archivo es el responsable de configurar y poner en funcionamiento todos los componentes del sistema.

Responsabilidades

- Crear la aplicación FastAPI.
- Registrar las rutas del Motor.
- Registrar las rutas del Simulador.
- Configurar políticas de seguridad.
- Iniciar el servidor web.

Importancia

Sin este archivo, el sistema no podría arrancar, ya que es el encargado de unir todos los módulos y hacerlos funcionar como una única aplicación.

CORS: COMUNICACIÓN ENTRE FRONTEND Y BACKEND

El Frontend y el Backend funcionan en puertos distintos.

Frontend → Puerto 5173

Backend → Puerto 8000

Por razones de seguridad, los navegadores bloquean por defecto la comunicación entre diferentes orígenes.

Para resolver este problema se utiliza CORS (Cross-Origin Resource Sharing).

CORS permite que el Frontend pueda realizar solicitudes al Backend sin que el navegador las bloquee.

8. EL FRONTEND: CÓMO ESTÁ ORGANIZADO REACT

Su objetivo es proporcionar una interfaz gráfica que permita al usuario construir redes lógicas, ejecutar simulaciones y visualizar resultados.

A diferencia del Backend, el Frontend no realiza cálculos lógicos. Su función consiste únicamente en mostrar información y capturar las acciones del usuario.

```
frontend/src/
├── App.tsx                ← cerebro: tiene todos los datos
├── compartido/            ← tipos, llamadas HTTP, colores
│   ├── tipos.ts
│   ├── api.ts
│   └── constantes.ts
├── editor/                ← todo lo que permite editar la red
│   ├── CanvasEditor.tsx
│   ├── FormularioNodo.tsx
│   └── FormularioArista.tsx
└── visualizador/          ← todo lo que muestra los resultados
    ├── ControlesSimulacion.tsx
    └── PanelResultados.tsx
```

Funciones principales

- Crear nodos.
- Crear conexiones.
- Visualizar resultados.
- Mostrar simulaciones.
- Interactuar con el Backend.

IPOSTS: DEFINICIÓN DE TIPOS

Su función es garantizar que la información tenga siempre el formato correcto.

Beneficios

- Reduce errores.
- Mejora la calidad del código.
- Facilita el mantenimiento.

```
export type ValorBelnap = 'T' | 'F' | 'B' | 'N';

export interface Nodo {
  id: Id;
  etiqueta: string;
  tipo: TipoNodo; // 'premisa' | 'AND' | 'OR' | 'XOR' | 'NOT' | 'IMP'
  propiedades: PropiedadesNodo;
  posicion: Posicion;
}
```

La lógica de Belnap
únicamente permite los
siguientes valores:

T

F

B

N

API.TS — CÓMO HABLA EL FRONTEND CON EL BACKEND

Importancia

Este archivo centraliza toda la comunicación con el servidor y evita que los demás componentes tengan que conocer detalles sobre HTTP.

```
export async function iniciarSimulacion(red: Red): Promise<EstadoSimulacion> {  
  const res = await fetch(`${BASE}/api/v1/simulaciones`, {
```

Proceso de comunicación

- 1.El usuario solicita una simulación.
- 2.El Frontend genera una petición HTTP.
- 3.El Backend recibe la información.
- 4.El Backend procesa la solicitud.
- 5.Se devuelve una respuesta.
- 6.El Frontend actualiza la interfaz.

```
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({ red }),    // objeto JS → JSON  
  });  
  if (!res.ok) throw new Error(`${res.status}`);  
  return res.json();    // JSON → objeto JavaScript  
}
```

APP.TSX — EL ESTADO CENTRAL

Aquí se almacena toda la información relevante de la aplicación.

Información administrada

- Lista de nodos.
- Lista de aristas.
- Estado de simulación.
- Paso visible actualmente.

```
const [nodos,      setNodos]      = useState<Node[]>([]);  
const [aristas,    setAristas]    = useState<Arista[]>([]);  
const [simulacion, setSimulacion] = useState<EstadoSimulacion | null>(null);  
const [pasoVisible, setPasoVisible] = useState(0);
```

CANVASEDITOR.TSX: EDITOR VISUAL

CanvasEditor es el componente encargado de dibujar la red lógica.

Su función es representar gráficamente nodos, conexiones y resultados.

Funciones principales

- Dibujar nodos.
- Dibujar aristas.
- Permitir arrastrar elementos.
- Actualizar posiciones.
- Mostrar cambios visuales.

```
const posicionesRef = useRef<Record<string, {x: number; y: number}>>({});
const draggingRef = useRef<{id: string; ox: number; oy: number} | null>(null);

// Los listeners se registran UNA SOLA VEZ:
useEffect(() => {
  const onMove = (e: MouseEvent) => {
    if (!draggingRef.current) return;
    // Actualizar el DOM directamente – sin re-render de React
    svg.querySelector(`[data-nid='${draggingRef.current.id}']`)
      ?.setAttribute('transform', `translate(${nx},${ny})`);
  };
  window.addEventListener('mousemove', onMove);
  return () => window.removeEventListener('mousemove', onMove);
}, [onMoverNodo]); // ← solo se registra una vez
```

9. FLUJO COMPLETO DE UNA SIMULACIÓN

Este apartado explica todo lo que ocurre dentro del sistema desde el momento en que el usuario presiona el botón "Simular Red" hasta que observa los resultados en pantalla.

Paso	Quién actúa	Qué hace
1	<u>App.tsx</u>	<u>simular()</u> se ejecuta, llama a <u>iniciarSimulacion(red)</u>
2	<u>api.ts</u>	<u>fetch</u> POST /api/v1/simulaciones con la red en JSON
3	simulador/api.py	<u>FastAPI</u> recibe, <u>Pydantic</u> valida, convierte JSON → Python
4	simulador/servicio.py	<u>validar_red(red)</u> — si hay errores, HTTP 422 al <u>frontend</u>
5	simulador/servicio.py	<u>gestor.crear()</u> → nuevo UUID, fase = CORRIENDO
6	motor/algoritmos.py	<u>_orden_topologico(red)</u> → [P, PQ, A, Q]
7	motor/algoritmos.py	Guarda estado inicial (operadores = N) como paso 0

8	motor/algoritmos.py	Iteración 1: $A = \text{AND_TABLE}[(T, T)] = T$, $Q = \text{OR}([T]) = T$
9	motor/algoritmos.py	Iteración 2: ningún valor cambia → convergido=True
10	simulador/servicio.py	<code>gestor.establecer_historial()</code> , fase = COMPLETADO
11	simulador/api.py	Serializa <code>EstadoSimulacion</code> completo → HTTP 201
12	<code>App.tsx</code>	<code>setSimulacion(res)</code> , <code>setPasoVisible(historial.length - 1)</code>
13	<code>CanvasEditor.tsx</code>	Nodos coloreados verde (T), flechas animadas
14	<code>PanelResultados.tsx</code>	Tarjetas $Q=T$, $A=T$, $P=T$, $PQ=T$ con colores

Importancia del Flujo Completo

Este proceso demuestra cómo interactúan todos los componentes del proyecto.

Sin este flujo:

- El Frontend no tendría información.
- El Motor no recibiría datos.
- El Simulador no podría guardar resultados.

10. LOS 5 PRINCIPIOS SOLID APLICADOS AL PROYECTO

¿Qué son los principios SOLID?

Son cinco principios de diseño orientado a objetos que ayudan a desarrollar software más organizado, mantenible y escalable.

S - SINGLE RESPONSIBILITY PRINCIPLE

Este principio establece que cada componente debe tener una única función.

Por ejemplo:

algoritmos.py

Su única responsabilidad es:

Calcular valores lógicos

No valida.

No dibuja.

No guarda historial.

Solo calcula.

validadores.py

Su única responsabilidad es:

Validar datos

No calcula.

No simula.

No dibuja.

Beneficio

Cuando ocurre un error es más fácil localizarlo.

Cada archivo tiene una función claramente definida.

O - OPEN CLOSED PRINCIPLE

El sistema permite agregar nuevas funcionalidades sin alterar gran parte del código existente.

Operadores:

AND

OR

NOT

XOR

IMP

Agregar:

NAND

Beneficio

Permite que el sistema
crezca sin generar errores
en módulos ya probados.

Solo sería necesario modificar los archivos relacionados con operadores.

El resto del sistema seguiría funcionando exactamente igual.

L - LISKOV SUBSTITUTION PRINCIPLE

Sustitución de Implementaciones

Este principio indica que una clase puede reemplazarse por otra siempre que cumpla el mismo contrato.

Beneficio

Permite reemplazar componentes sin modificar el resto de la aplicación.

I - INTERFACE SEGREGATION PRINCIPLE

Interfaces Especializadas

No todos los componentes necesitan las mismas funciones.

Por ello existen interfaces diferentes.

IMotorCalculo

Solo calcula resultados finales.

IMotorCalculoIterativo

Calcula además el historial completo.

Ventaja

Cada componente utiliza únicamente los métodos que realmente necesita.

Esto reduce complejidad y dependencias innecesarias.

D - DEPENDENCY INVERSION PRINCIPLE

Dependencia de Abstracciones

Este principio establece que los módulos no deben depender de implementaciones concretas.

Deben depender de interfaces.

¿Por qué es importante?

Porque permite cambiar la implementación sin afectar a los demás componentes.

El sistema se vuelve más flexible y fácil de mantener.

II. PRUEBAS AUTOMÁTICAS (TESTS)

¿Qué son los tests?

Los tests son programas diseñados para verificar automáticamente que el sistema funciona correctamente. Permiten detectar errores antes de que el usuario los encuentre.

¿Por qué son importantes?

Imagina que modificamos el operador AND.

Sin pruebas:

No sabemos si rompimos algo.

Con pruebas:

El sistema lo detecta inmediatamente.

TESTS DEL MOTOR (SIN MOCKS)

Comprueban que las tablas de verdad produzcan los resultados correctos.

```
class TestAND:
    def test_T_AND_T_es_T(self): assert AND_TABLE[("T","T")] == "T"
    def test_T_AND_F_es_F(self): assert AND_TABLE[("T","F")] == "F"
    # ... 14 casos más

class TestPropagacion:
    def test_premisa_T_se_propaga_por_AND(self):
        r = obtener_motor().calcular(_red_cadena())
        assert r.valores_nodos["nodo-A"] == "T"
```


TESTS DEL SIMULADOR (CON MOCKS)

Comprueban:

- Creación de simulaciones.
- Almacenamiento de historial.
- Avance de pasos.
- Gestión de estados.

Beneficios

Las pruebas permiten:

- Detectar errores rápidamente.
- Facilitar mantenimiento.
- Validar nuevas funcionalidades.
- Aumentar confiabilidad del sistema.

```
def _servicio_con_motor_mock(pasos=None):  
    motor = MagicMock(spec=IMotorCalculoIterativo)  
    motor.calcular_pasos.return_value = pasos or _pasos_mock()  
    ...  
  
# El Simulador acepta el mock porque solo conoce IMotorCalculoIterativo.  
# Si estos tests fallan, el problema está en el Simulador, no en el Motor.
```

12. RESUMEN GENERAL DE LA ARQUITECTURA

¿Cómo trabajan juntos todos los componentes?

El proyecto puede entenderse como una cadena de responsabilidades.

Compartido

Define:

- Nodos.
- Aristas.
- Redes.
- Contratos.
- Validaciones.

Archivo	Lo que hace con los datos
<code>compartido/modelos.py</code>	Define las cajas: cómo se ve un Nodo, una Arista, una Red
<code>compartido/contratos.py</code>	Define las promesas: qué métodos debe tener el Motor y el Simulador
<code>compartido/validadores.py</code>	Revisa que los datos tienen sentido antes de calcular
<code>motor/algoritmos.py</code>	Consulta las tablas y produce el historial de valores
<code>motor/api.py</code>	Recibe JSON, llama al Motor, devuelve JSON
<code>simulador/estado.py</code>	Guarda el historial en memoria y mueve el puntero de paso
<code>simulador/servicio.py</code>	Coordina: valida → llama al Motor → guarda → devuelve ID
<code>simulador/api.py</code>	Recibe JSON, llama al Servicio, conecta Motor con Simulador
<code>principal.py</code>	Enciende el servidor y conecta todos los routers
<code>compartido/tipos.ts</code>	Define las mismas cajas que <code>modelos.py</code> pero en TypeScript
<code>compartido/api.ts</code>	Envía solicitudes HTTP al backend y devuelve los datos tipados
<code>compartido/constantes.ts</code>	Guarda los colores, presets y paleta sin lógica

<code>editor/FormularioNodo.tsx</code>	Captura lo que el usuario quiere crear y llama a <code>onAgregar</code>
<code>editor/FormularioArista.tsx</code>	Captura origen/destino (bloquea premisas como destino)
<code>editor/CanvasEditor.tsx</code>	Dibuja el grafo en SVG y maneja el drag-and-drop con refs
<code>visualizador/ControlesSimulacion.tsx</code>	Muestra botones Play/Pausar/⏮/⏭ y llama a callbacks del padre
<code>visualizador/PanelResultados.tsx</code>	Muestra los valores T/F/B/N por nodo y la leyenda de colores
<code>App.tsx</code>	Tiene todos los datos, pasa props a todos, llama a la API, navega el historial

CONCLUSIÓN

EPiC Playground es un sistema diseñado para simular la propagación de información utilizando la lógica de Belnap. Su arquitectura separa claramente las responsabilidades entre Frontend, Backend, Motor y Simulador, permitiendo construir una aplicación organizada, mantenible y escalable. Además, la aplicación incorpora principios SOLID, validación de datos, pruebas automatizadas y una arquitectura cliente-servidor, convirtiéndose en un ejemplo completo de desarrollo de software aplicado a sistemas lógicos.